AWS Project: Secure & Scalable Web Application

A Production-Grade Infrastructure Project • Revised V2 Architecture

Project Overview

This project combines Networking & Security, Auto Scaling, and IAM Identity & Permissions into a single, production-grade AWS infrastructure. It mirrors how real companies build and secure their cloud environments, emphasizing modern access patterns, cost-optimization, and Infrastructure as Code (IaC).

By completing this, you will cover roughly 60-70% of the AWS Solutions Architect Associate exam topics while generating a highly demonstrable portfolio piece.

Architecture Traffic Flow

A user's request passes through multiple layers of defense before hitting your application:

```
User → CloudFront + WAF → ALB (public subnet) → EC2 ASG (private subnet) → RDS (private subnet)
```

IAM acts as the invisible backbone — every service-to-service communication uses roles and policies, never hardcoded credentials.

Architecture Layers

Layer 1 — Edge Protection (WAF + CloudFront)

- **CloudFront distribution:** Positioned in front of all application resources for global caching.
- **WAF rules:** Configured for rate limiting, IP blocking, and SQL injection / XSS protection.
- **Geo-restriction:** Enabled to allow traffic only from authorized countries.

Layer 2 — Cost-Optimized Custom VPC & Networking

- VPC deployed with public and private subnets distributed across 2 Availability Zones.
- **VPC Endpoints (Cost Optimization):** Instead of relying on an expensive NAT Gateway (~$32/mo), the architecture utilizes VPC Interface Endpoints for SSM and Secrets Manager, and a Gateway Endpoint for S3. This allows private instances to reach necessary AWS services securely and cost-effectively without traversing the public internet.
- **SSM Session Manager:** Used for private EC2 access, eliminating the need for a bastion host, SSH keys, or port 22 exposure.
- **Security Groups:** Utilizing security group chaining (e.g., the EC2 group only accepts ingress traffic from the ALB security group).

Layer 3 — Auto Scaling App Layer

- EC2 Launch Template containing your web application.
- Auto Scaling Group (ASG) deployed across both private subnets for resilience.
- Application Load Balancer (ALB) in the public subnet distributing traffic.
- **Dynamic Scaling:** Scale-out policy triggers when CPU > 60%; Scale-in policy triggers when CPU < 20%.

> **Architectural Note: Stateless Application Design**
>
> Because the ASG will dynamically add and remove EC2 instances, the application must be entirely **stateless**. User sessions should be managed externally via Amazon ElastiCache (Redis) or DynamoDB to ensure users are not logged out during a scale-in event.

Layer 4 — Data Layer

- RDS (MySQL or Postgres) deployed strictly in a private subnet.
- Database is only accessible from the EC2 security group — it is never publicly routable.
- Credentials are dynamically fetched via Secrets Manager, never hardcoded.

> **Architectural Note: RDS High Availability**
>
> For this portfolio project, RDS is deployed in a **Single-AZ** configuration to optimize costs. However, it should be heavily documented that a true production deployment would mandate a **Multi-AZ** configuration for synchronous replication and automatic failover.

Layer 5 — IAM Identity & Permissions

EC2 IAM Role Permissions

An IAM Role is attached to the Launch Template. The instances receive temporary, rotating credentials automatically, scoped to the principle of least privilege.

Permission Action	Architectural Purpose
`s3:GetObject` / `s3:PutObject`	Read/write application assets to a specific, restricted S3 bucket.
`secretsmanager:GetSecretValue`	Fetch DB password — strictly scoped to the exact secret ARN.
`cloudwatch:PutMetricData`	Push custom application metrics for monitoring and scaling alarms.
`ssm:GetParameter`	Read configuration values securely from Parameter Store.
`ssm:StartSession`	Enable SSM access via *AmazonSSMManagedInstanceCore* instead of SSH.

Team IAM Users & Groups

To simulate a real enterprise environment, create IAM users and assign them to groups with specific permission boundaries to prevent privilege escalation.

Simulated User	IAM Group	Access Level & Capabilities
`dev-alice`	Developers	EC2 read-only, S3 read/write, CloudWatch logs access.
`ops-bob`	Operations	EC2 full access, ASG management, ALB config, RDS read.
`admin-carol`	Administrators	Full access except for IAM user management and billing.
`readonly-dave`	Auditors	SecurityAudit managed policy; ReadOnly on all infrastructure.

Secrets Manager + Scoped IAM Policy

The EC2 role is granted access to Secrets Manager, but the policy is scoped specifically to the database secret ARN to prevent horizontal credential access:

```
{ "Effect": "Allow", "Action": "secretsmanager:GetSecretValue", "Resource": "arn:aws:secretsmanager:us-east-1:123456789:secret:myapp/db-password" }
```

Core Requirement: Infrastructure as Code (IaC)

While building this via the AWS Management Console is a great learning exercise, **the final portfolio submission must be scripted.** Use Terraform or AWS CDK to define this entire architecture. Demonstrating state management, modular code structure, and reproducible environments is a hard requirement for modern Cloud Engineering roles.

Recommended Build Order & Explanations

Follow this sequence to build and test the project incrementally. Understanding the "why" behind this order is critical for troubleshooting.

1. VPC & Cost-Optimized Networking (Layer 2)

What to do: Create a custom VPC with public and private subnets. Instead of provisioning an expensive NAT Gateway, configure VPC Endpoints (Interface endpoints for SSM and Secrets Manager, and a Gateway endpoint for S3).

Why it's important: *The network is your application's foundation. Public subnets are for internet-facing resources (like load balancers), while private subnets hide your servers and databases. Swapping the NAT Gateway for VPC Endpoints saves ~$32/month and keeps all your AWS API traffic securely within the AWS private network.*

2. IAM Roles & Policies (Layer 5)

What to do: Create the EC2 IAM Role, user groups, and custom least-privilege policies.

Why it's important: *IAM is the invisible security backbone of AWS. By setting this up before the compute resources, you ensure your EC2 instances boot up with the exact permissions they need (like permission to read the database password) without ever requiring hardcoded credentials.*

3. Security Groups

What to do: Create security groups for the Application Load Balancer (ALB), EC2 instances, and RDS database using "security group chaining."

Why it's important: *Chaining means setting rules based on other security groups rather than IP addresses. For example, your RDS security group should only accept inbound traffic from the EC2 security group. This ensures the database is physically unreachable from anywhere except your web servers.*

4. Secrets Manager

What to do: Store your RDS database credentials here.

Why it's important: *Hardcoding passwords in application code is a critical security vulnerability. Using Secrets Manager allows your application to fetch the password dynamically at runtime.*

5. RDS Database (Layer 4)

What to do: Deploy an RDS instance (MySQL/Postgres) in the private subnet, configured to pull credentials from Secrets Manager.

Why it's important: *Placing the database in a private subnet ensures it is entirely isolated from the internet. The database must exist before the application boots. (Remember: Use Single-AZ for this portfolio project to save costs, but note that production requires Multi-AZ).*

6. EC2, Launch Templates, & Auto Scaling (Layer 3)

What to do: Create a Launch Template for your stateless web app (with the IAM role attached and SSM agent enabled), then attach it to an Auto Scaling Group (ASG) in the private subnets.

Why it's important: *The Launch Template standardizes your server configuration. Because your app is stateless, the ASG can safely add servers when CPU usage spikes and remove them when traffic drops, maximizing performance while minimizing costs. SSM Session Manager allows you to access these instances securely without opening SSH port 22.*

7. ALB, CloudFront, & WAF (Layer 1)

What to do: Set up the ALB in the public subnet to route traffic to the ASG. Then, place a CloudFront distribution in front of the ALB and attach a Web Application Firewall (WAF).

Why it's important: *This is your edge protection. CloudFront caches content globally to reduce latency. The WAF inspects incoming traffic and blocks malicious activity (like SQL injections or DDOS attempts) before it even reaches your load balancer.*

8. Infrastructure as Code Translation

What to do: Once you understand how to build this manually in the AWS Console, script the entire environment using Terraform or AWS CDK.

Why it's important: *IaC allows you to version-control your infrastructure, tear it down to save money, and rebuild it perfectly in minutes. Manual console configuration is prone to human error and difficult to replicate.*

Skills & Concepts Demonstrated

Domain	Concepts Practiced
Networking	VPC, public/private subnets, VPC Endpoints, route tables, ALB
Security	WAF rules, security group chaining, least-privilege IAM design
Identity	IAM roles, users, groups, custom policies, permission boundaries
Compute & State	EC2, Launch Templates, ASGs, stateless app design (ElastiCache)
Access Control	SSM Session Manager (zero-trust pattern, no SSH/bastion)
Database	RDS in private subnets, Single vs Multi-AZ architecture
Edge Services	CloudFront distribution, WAF rate limiting & geo-restriction

Portfolio & Interview Tips

- **Publish to GitHub:** Include a clear README, instructions to run your IaC code, and an architecture diagram (using tools like Draw.io or Lucidchart).
- **Document Decisions:** Explain *why* you chose VPC Endpoints over a NAT Gateway (cost) and Single-AZ over Multi-AZ (cost vs. prod). Employers look for engineers who understand trade-offs.
- **Video Walkthrough:** Record a 3-minute video showing the auto-scaling in action via a load test (using a tool like Apache JMeter or Artillery).
- **Highlight Security:** Be prepared to talk extensively about why SSM is better than SSH and how Secrets Manager protects against credentials leaking in GitHub repositories.

Estimated Monthly Costs (Cost-Optimized)

By replacing the NAT Gateway with VPC Endpoints, the baseline cost of this project is significantly reduced. This proves to prospective employers that you design with budget constraints in mind.

AWS Resource	Approximate Monthly Cost
EC2 (t3.micro x2, Auto Scaling)	~$15–20
Application Load Balancer (ALB)	~$16
RDS (db.t3.micro, single-AZ)	~$13
VPC Interface Endpoints (SSM, Secrets)	~$15 (S3 Gateway Endpoint is Free)
CloudFront + WAF	~$5–10
Secrets Manager, CloudWatch, S3	~$1–2

Cost Management Tip

Always execute `terraform destroy` (or your equivalent IaC tear-down command) when you are done developing for the day. Set up a billing alert in AWS Budgets at $20 to avoid any surprise charges.